

Clase 2 — Introducción al manejo de R con RStudio

Diplomatura en Análisis Exploratorio de Datos Espaciales aplicados a las Ciencias Sociales

Dr. David Schomwandt

Nora Lucioni

2026-06-19

Contenidos

1. Introducción	2
2. Variables y expresiones	2
3. Vectores	2
3.1. Tipos de vectores	2
3.2. Operaciones con vectores	3
3.3. Acceso a elementos	3
4. Matrices	4
4.1. Operaciones con matrices	4
5. Listas	4
5.1. Acceso a elementos de una lista	5
5.2. Agregar elementos con <code>append()</code>	5
6. Tuplas (vectores nombrados)	5
7. DataFrames	6
7.1. Creación de un DataFrame	6
7.2. Exploración del DataFrame	6
7.3. Acceso y manipulación	6
7.4. Agregar columnas con <code>cbind()</code>	7
8. Importación de datos	7
8.1. Desde CSV con <code>readr</code>	7
8.2. Desde XLSX con <code>xlsx</code> y <code>rJava</code>	7
8.3. Desde URL	8
8.4. Desde JSON	8
9. Resumen de estructuras de datos	9
10. Ejercicios prácticos	9
10.1. Ejercicio 1: Vectores y estadísticas	9
10.2. Ejercicio 2: DataFrame desde cero	9
10.3. Ejercicio 3: Importación y exploración	9
11. Recursos adicionales	10

1. Introducción

En esta clase nos familiarizamos con los **tipos de datos y estructuras fundamentales** de R, y aprendemos a importar datasets desde distintas fuentes (archivos locales, URLs y APIs). R es un lenguaje de programación especialmente diseñado para el análisis estadístico y la visualización de datos, ampliamente utilizado en ciencias sociales, geografía y disciplinas afines.

Requisito previo: tener instalados R (4.0) y RStudio. Los paquetes adicionales se instalan a lo largo de la clase.

2. Variables y expresiones

En R, el operador de asignación es `<-`. También puede usarse `=`, pero la convención en la comunidad R es preferir `<-`.

```
# Asignación de variables
nombre <- "Buenos Aires"
poblacion <- 3120612
tasa_desempleo <- 8.5
es_capital <- TRUE

# Imprimir valores
nombre
poblacion
```

Las variables pueden ser de distintos **tipos básicos**:

Tipo	Ejemplo	Descripción
character	"Buenos Aires"	Texto (cadena de caracteres)
numeric	3120612	Número entero o decimal
double	8.5	Número de punto flotante
logical	TRUE / FALSE	Valor lógico booleano

```
# Verificar el tipo de una variable
class(nombre)
class(poblacion)
class(tasa_desempleo)
class(es_capital)
```

3. Vectores

Un **vector** es la estructura de datos más básica en R: una secuencia ordenada de elementos del mismo tipo. Se crea con la función `c()` (*combine*).

3.1. Tipos de vectores

```
# Vector numérico (enteros)
comunas <- c(1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15)
```

```
# Vector de decimales
superficies_km2 <- c(5.27, 2.76, 6.31, 9.17, 6.85, 6.61, 12.43,
                    21.37, 16.53, 13.69, 13.43, 7.44, 14.05,
                    13.14, 12.44)

# Vector de texto (character)
nombres_comunas <- c("San Nicolás-Monserrat", "Recoleta", "Balvanera-San Cristóbal",
                    "La Boca-Barracas-Parque Patricios-Nueva Pompeya",
                    "Almagro-Boedo", "Caballito", "Flores-Parque Chacabuco",
                    "Villa Soldati-Villa Riachuelo-Villa Lugano",
                    "Liniers-Mataderos-Parque Avellaneda",
                    "Floresta-Monte Castro-Vélez Sársfield-Versalles-Villa Real-Villa del P",
                    "Villa General Mitre-Villa Devoto-Villa del Parque-Villa Santa Rita",
                    "Coghlan-Saavedra-Villa Urquiza-Villa Pueyrredón",
                    "Belgrano-Núñez-Colegiales",
                    "Palermo",
                    "Chacarita-Villa Crespo-Paternal-Villa Ortúzar-Agronomía-Parque Chas")

# Vector lógico
es_costero <- c(TRUE, FALSE, FALSE, TRUE, FALSE, FALSE, FALSE,
                TRUE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE)
```

3.2. Operaciones con vectores

```
# Longitud del vector
length(comunas)

# Suma y estadísticas
sum(superficies_km2)
max(superficies_km2)
mean(superficies_km2)
median(superficies_km2)

# Ordenar
sort(superficies_km2)           # ascendente
sort(superficies_km2, decreasing = TRUE) # descendente

# Índice del orden
order(superficies_km2)
```

3.3. Acceso a elementos

```
# Acceso por posición (R indexa desde 1)
superficies_km2[1]           # primera comunas
superficies_km2[c(1,3)]      # primera y tercera
superficies_km2[1:5]         # primeras cinco

# Acceso con condición lógica
superficies_km2[superficies_km2 > 10]
```

4. Matrices

Una **matriz** es una estructura bidimensional (filas y columnas) donde todos los elementos son del mismo tipo. Se crea con `matrix()`.

```
# Crear una matriz de datos de población por grupo etario (inventado)
datos_poblacion <- matrix(
  data = c(120000, 115000, 98000, 87000,
           200000, 195000, 180000, 170000,
           150000, 145000, 130000, 120000),
  nrow = 3,
  ncol = 4,
  byrow = TRUE
)

# Asignar nombres a filas y columnas
rownames(datos_poblacion) <- c("0-17 años", "18-64 años", "65+ años")
colnames(datos_poblacion) <- c("Norte", "Sur", "Este", "Oeste")

datos_poblacion
```

4.1. Operaciones con matrices

```
# Dimensiones
nrow(datos_poblacion)
ncol(datos_poblacion)
dim(datos_poblacion)

# Medias por fila y columna
rowMeans(datos_poblacion) # promedio por grupo etario
colMeans(datos_poblacion) # promedio por zona

# Sumas
rowSums(datos_poblacion) # total por grupo etario
colSums(datos_poblacion) # total por zona

# Acceso a elementos: [fila, columna]
datos_poblacion[1, 2] # fila 1, columna 2
datos_poblacion[, 1]  # toda la columna 1 (zona Norte)
datos_poblacion[2, ]  # toda la fila 2 (18-64 años)
```

5. Listas

Una **lista** puede contener elementos de distintos tipos (incluso otras listas o dataframes). Es la estructura más flexible de R.

```
# Crear una lista con información de un barrio
barrio_palermo <- list(
```

```

nombre      = "Palermo",
comuna      = 14,
superficie  = 15.98,
poblacion   = 234184,
es_costero  = FALSE,
limites     = c("Belgrano", "Recoleta", "Villa Crespo", "Chacarita")
)

barrio_palermo

```

5.1. Acceso a elementos de una lista

```

# Por nombre con $
barrio_palermo$nombre
barrio_palermo$poblacion

# Por índice con [[]]
barrio_palermo[[1]]  # primer elemento
barrio_palermo[[4]]  # cuarto elemento

# Modificar un elemento
barrio_palermo$poblacion <- 235000

```

5.2. Agregar elementos con append()

```

barrio_palermo <- append(
  barrio_palermo,
  list(densidad = barrio_palermo$poblacion / barrio_palermo$superficie)
)

barrio_palermo$densidad

```

6. Tuplas (vectores nombrados)

R no tiene un tipo nativo “tupla” como Python, pero podemos simularlas con **vectores nombrados**, que son inmutables por convención.

```

# Coordenadas de la Plaza de Mayo (latitud, longitud)
plaza_mayo <- c(lat = -34.6083, lon = -58.3712)

plaza_mayo
plaza_mayo["lat"]
plaza_mayo["lon"]

# Información de la ciudad
info_caba <- c(
  nombre      = "Ciudad Autónoma de Buenos Aires",
  pais        = "Argentina",
  fundacion   = "1580"
)

```

```
)

info_caba["nombre"]
```

7. DataFrames

El **DataFrame** es la estructura central para el análisis de datos en R: una tabla con filas (observaciones) y columnas (variables), donde cada columna puede ser de un tipo diferente. Es equivalente a una hoja de cálculo o a una tabla de base de datos.

7.1. Creación de un DataFrame

```
# DataFrame con datos de comunas de CABA
comunas_df <- data.frame(
  id_comuna = 1:5,
  nombre    = c("Commune 1", "Commune 2", "Commune 3", "Commune 4", "Commune 5"),
  superficie = c(5.27, 2.76, 6.31, 9.17, 6.85),
  poblacion  = c(205886, 157932, 187537, 218245, 179005),
  es_costero = c(TRUE, FALSE, FALSE, TRUE, FALSE)
)
```

7.2. Exploración del DataFrame

```
# Estructura y tipos
str(comunas_df)

# Resumen estadístico
summary(comunas_df)

# Dimensiones
nrow(comunas_df) # número de filas
ncol(comunas_df) # número de columnas
dim(comunas_df)  # ambas dimensiones

# Nombres de columnas
colnames(comunas_df)
```

7.3. Acceso y manipulación

```
# Acceso por columna
comunas_df$nombre
comunas_df[["poblacion"]]

# Acceso por fila y columna [fila, columna]
comunas_df[1, ]           # primera fila completa
comunas_df[, "nombre"]    # columna "nombre"
comunas_df[2, "superficie"]
```

```
# Filtrado condicional
comunas_df[comunas_df$es_costero == TRUE, ]
comunas_df[comunas_df$poblacion > 190000, ]
```

7.4. Agregar columnas con cbind()

```
# Calcular densidad poblacional
densidad <- comunas_df$poblacion / comunas_df$superficie

comunas_df <- cbind(comunas_df, densidad_hab_km2 = densidad)
head(comunas_df)
```

8. Importación de datos

8.1. Desde CSV con readr

El paquete `readr` (parte del *tidyverse*) ofrece funciones rápidas y robustas para leer archivos de texto delimitado.

```
# Instalar y cargar readr (si no está instalado)
# install.packages("readr")
library(readr)

# Leer CSV local
barrios <- read_csv("datos/barrios_comunas_p_Ciencia_de_Datos_y_PP.csv")

# Vista rápida
head(barrios)
str(barrios)
```

También puede usarse la función base de R:

```
# Con función base
barrios_base <- read.csv(
  "datos/barrios_comunas_p_Ciencia_de_Datos_y_PP.csv",
  encoding = "UTF-8",
  stringsAsFactors = FALSE
)
```

Diferencia clave: `readr::read_csv()` es más rápida, infiere mejor los tipos de columnas y no convierte strings a factores por defecto.

8.2. Desde XLSX con xlsx y rJava

Para leer archivos de Excel (.xlsx) necesitamos el paquete `xlsx`, que depende de Java.

```
# Instalar paquetes (solo la primera vez)
# install.packages("rJava")
# install.packages("xlsx")

library(rJava)
library(xlsx)
```

```
# Leer archivo Excel
poblacion_barrios <- read.xlsx(
  "datos/caba_pob_barrios_2010.xlsx",
  sheetIndex = 1
)

head(poblacion_barrios)
```

Nota: rJava requiere tener Java instalado en el sistema y que la versión (32/64 bits) coincida con la de R. Una alternativa sin dependencia de Java es `readxl::read_excel()`.

8.3. Desde URL

R puede leer datos directamente desde internet usando una URL como si fuera un archivo local.

```
# URL del dataset de COVID-19 (GitHub)
url_covid <- paste0(
  "https://raw.githubusercontent.com/eparker12/nCoV_tracker/",
  "master/input_data/coronavirus.csv"
)

covid_data <- read.csv(url(url_covid))

# Explorar
dim(covid_data)
head(covid_data[, 1:6])
```

8.4. Desde JSON

El formato JSON es muy común en APIs y datos web. Dos paquetes populares: `rjson` y `jsonlite`.

```
# Instalar paquetes
# install.packages("rjson")
# install.packages("jsonlite")

# --- Con rjson ---
library(rjson)

url_ny <- "https://data.ny.gov/api/views/9a8c-vfzj/rows.json"
datos_ny_raw <- fromJSON(file = url_ny)

# Los datos JSON suelen venir en listas anidadas
class(datos_ny_raw)
names(datos_ny_raw)

# --- Con jsonlite (más conveniente para APIs) ---
library(jsonlite)

datos_ny <- fromJSON(url_ny)

# jsonlite intenta convertir automáticamente a data.frame
```



```
class(datos_ny)
```

¿Cuándo usar cada uno? `jsonlite` es preferible cuando el JSON tiene estructura tabular (listas de objetos con los mismos campos), ya que lo convierte automáticamente a `DataFrame`. `rjson` es más flexible para JSONs con estructuras irregulares.

9. Resumen de estructuras de datos

Estructura	Dimensiones	Tipos de datos	Función de creación
Vector	1D	Homogéneo	<code>c()</code>
Matriz	2D	Homogéneo	<code>matrix()</code>
Lista	1D (anidado)	Heterogéneo	<code>list()</code>
Vector nombrado	1D	Homogéneo	<code>c(nombre = valor, ...)</code>
<code>DataFrame</code>	2D	Heterogéneo	<code>data.frame()</code>

10. Ejercicios prácticos

10.1. Ejercicio 1: Vectores y estadísticas

Crear un vector con la superficie en km^2 de los 48 barrios de CABA e identificar el barrio más grande y el más pequeño.

```
# Tu código aquí
superficies_barrios <- c(...)

max_superficie <- max(superficies_barrios)
min_superficie <- min(superficies_barrios)
```

10.2. Ejercicio 2: `DataFrame` desde cero

Construir un `DataFrame` con al menos 5 barrios de CABA que incluya: nombre del barrio, comuna, superficie y población (Censo 2010). Calcular la densidad poblacional y agregarla como nueva columna.

```
# Tu código aquí
mi_df <- data.frame(
  barrio    = c(...),
  comuna    = c(...),
  sup_km2   = c(...),
  poblacion = c(...)
)

mi_df$densidad <- ...
```

10.3. Ejercicio 3: Importación y exploración

1. Importar el dataset `barrios_comunas_p_Ciencia_de_Datos_y_PP.csv`.
2. Usar `str()` y `summary()` para explorar la estructura.

3. Filtrar los barrios que pertenecen a la Comuna 1.
4. Calcular la cantidad de barrios por comuna.

```
# Tu código aquí
library(readr)
barrios <- read_csv("datos/barrios_comunas_p_Ciencia_de_Datos_y_PP.csv")

# Explorar
str(barrios)

# Filtrar
barrios_c1 <- barrios[barrios$COMUNA == 1, ]

# Contar por comuna
table(barrios$COMUNA)
```

11. Recursos adicionales

- [Documentación oficial de R](#)
- [RStudio Cheatsheets](#) — hojas de referencia rápida
- [R for Data Science \(Wickham & Grolemund\)](#) — libro gratuito en línea
- [Data Buenos Aires](#) — portal de datos abiertos de CABA
- [INDEC](#) — datos censales de Argentina